

Hierarchical Detection of Malicious Command Lines Using Random Forest and Context-Aware LSTM

Yixian Jiang¹[0009-0005-9180-9757], Qinzhen Hu¹[0009-0009-1544-0432], and
Pavandeep Singh Baxi¹[0009-0008-0047-1925]

¹ School of Computing Science, University of Glasgow, Glasgow, G12 8RZ, UK

Yixian Jiang and Qinzhen Hu contributed equally to the manuscript.

Abstract. Malicious command-line activity remains a key vector in host-based cyber attacks, particularly when obfuscation and multi-step execution are used to evade detection. Existing approaches either rely on rule-based methods that are sensitive to variation or apply machine learning to isolated commands without incorporating temporal context. This paper presents a hierarchical detection framework that integrates feature-based machine learning with sequence-aware modeling within a unified evaluation setting. At the single-command level, we compare rule-based detection, hybrid logistic regression, and random forest classifiers over a shared feature representation. At the sequence level, these models are extended using LSTM-based context modeling to capture temporal dependencies across command streams. Experimental results show that random forest achieves the strongest performance for single-command detection, while the RF+LSTM model provides the best performance for command-stream analysis. These findings demonstrate that effective command-line threat detection benefits from combining strong local classification with context-aware refinement, particularly when malicious behavior unfolds across multiple steps.

Keywords: Command-Line Attack Detection · Random Forest · LSTM · Contextual Modeling · Malicious Command Recognition.

1 Introduction

Command-line interfaces (CLIs), such as bash, sh, PowerShell, and cmd, remain essential for system administration and routine host-side operations. At the same time, they are frequently abused in modern intrusions because they provide direct access to built-in system capabilities and can be used to execute downloading, decoding, persistence, and post-exploitation actions without relying on obvious standalone malware. This challenge is especially visible in Living-off-the-Land activity, where adversaries misuse legitimate system tools to blend malicious behavior into normal operational traces, and in command obfuscation, where encoding, string splitting, and syntax manipulation are used to evade signature-based detection [1, 2]. As a result, distinguishing legitimate command usage from malicious command execution has become an important problem in host-side threat detection.

Existing research on command-line security largely follows two directions. The first is rule-based detection, which relies on predefined signatures, suspicious

argument patterns, or regular expressions. These methods are attractive because they are interpretable and often precise for known attack patterns, but they are also brittle when adversaries introduce obfuscation or previously unseen variants [1, 2]. The second direction is single-command machine learning classification, where each command or script is classified independently using textual, statistical, or structural features. Prior work on malicious PowerShell detection has shown that machine learning and deep learning methods can improve robustness beyond fixed rules, including neural approaches for malicious command detection, feature-based malicious script detection, and more recent efforts that explicitly target robustness under code mixing and obfuscation [3–5]. However, these approaches still focus primarily on isolated commands or scripts, which limits their ability to represent how malicious behavior unfolds over time.

This limitation matters because real attacks rarely appear as a single obviously malicious command. In practice, many incidents unfold as multi-step command sequences, where the risk of a given step becomes clear only when viewed together with preceding and subsequent actions. A decoding or interpreter invocation, for example, may appear benign in isolation but become suspicious when it follows a download step and precedes execution or lateral-movement behavior. This broader point is consistent with sequence-based anomaly detection research in security logs, where LSTM-based approaches and later studies emphasize that temporally ordered event sequences often reveal patterns that are missed by static, per-event analysis [6, 7]. For command-line threat detection, this demonstrates that an effective defense should combine single-command discrimination with context-aware sequence modeling, rather than relying on either one alone.

To address this limitation, this paper proposes a hierarchical command-line detection framework that integrates static command classification with sequence-aware analysis. In the first layer, we evaluate rule-based detection, hybrid logistic regression, and random forest models for single-command classification. In the second layer, these models are extended with LSTM-based context modeling to support command-stream detection.

This work aligns with the growing use of machine learning in cybersecurity, particularly in settings where adversarial behavior evolves over time. The proposed framework reflects a shift toward context-aware AI-based detection, where temporal dependencies are incorporated alongside static feature representations.

The main contribution of this work is a unified hierarchical evaluation framework that enables direct comparison between static command classification and context-aware command-stream detection. The study further provides empirical insight into how detection performance changes when temporal context is introduced, highlighting the conditions under which sequence modeling improves detection of multi-step malicious behavior. This is particularly relevant for host-based monitoring systems, where command-line activity represents a primary interface for both legitimate administration and adversarial operations.

2 Related Work

2.1 Rule-based Command Detection

Malicious command-line detection has long relied on rule knowledge, suspicious pattern matching, and expert-crafted indicators. This remains especially relevant in Living-off-the-Land (LotL) settings, where attackers abuse native system tools and binaries to make malicious activity resemble normal administrative behavior. In such scenarios, rule-based reasoning is still useful because it is transparent and effective for known patterns. At the same time, recent work shows that fixed signatures are often fragile when commands are reformulated, obfuscated, or embedded in more varied operational contexts. In response, recent research has increasingly moved toward stronger command-line representations rather than relying only on surface-form matching.

This shift is reflected in several recent directions. Zhu et al. proposed a LotL-oriented detection system that combines expert knowledge with feature-based learning, showing continued interest in integrating domain knowledge with data-driven detection [8]. At the representation level, Huang et al. introduced CmdCaliper, a semantic-aware command-line embedding model and dataset designed for security research, showing that command lines benefit from task-specific representations rather than generic text embeddings [9]. Hussain et al. further explored this direction by training a language model to learn the syntax of commands, suggesting that command structure itself can provide useful information for distinguishing different forms of command behavior [10]. Together, these studies show that recent command-line research is moving beyond purely surface-level indicators and toward representations that better capture syntax, semantics, and operational intent.

2.2 ML-based Command Classification

A second recent direction focuses on improving the robustness of single-command or single-script malicious detection using machine learning and deep learning. Much of this work is motivated by the difficulty of handling obfuscation, code mixing, and structurally complex scripts, especially in PowerShell environments. Instead of depending on rigid signatures, these studies attempt to recover or learn more stable signals from the command or script itself.

Recent work reflects several complementary strategies. Li et al. proposed PowerPeeler, a dynamic deobfuscation method for PowerShell scripts, with the objective of exposing malicious logic before downstream analysis [11]. Fu et al. addressed robustness under code mixing and obfuscation by combining sentence-transformer-based representations with similarity learning, emphasizing semantic resilience rather than exact lexical matching [5]. Zhang et al. proposed Power-ASTNN, which combines deobfuscation with AST-based neural modeling to improve malicious PowerShell script detection through structural program information [12]. Although these methods differ in representation and modeling strategy, they share a common strength: they improve robustness beyond fixed

signatures and better capture malicious characteristics in individual commands or scripts.

At the same time, this literature also shares an important limitation. Most of these approaches are still evaluated at the level of isolated commands or individual scripts. As a result, they are effective for static malicious content recognition, but they generally do not model how suspicious behavior unfolds across a sequence of related commands. For real host-side attacks, where malicious intent may emerge gradually across multiple steps, this leaves an important part of the detection problem unresolved.

2.3 Sequence-based Context Modeling

Alongside advances in static command and script detection, recent work in security analytics has continued to emphasize the value of sequence-aware modeling. This line of research is usually framed around security logs, event streams, or authentication traces rather than command lines specifically, but its central insight is highly relevant: the meaning of a security event often depends on its temporal position and relation to surrounding events. For this reason, recent studies increasingly use sequence models to analyze ordered event data instead of treating each event as an independent instance.

For example, Gökstorp et al. studied anomaly detection in security logs using sequence modeling, showing that ordered event structure can support more effective detection than purely static analysis [13]. Chai et al. similarly modeled log-event sequences through template and parameter parsing with BERT-based methods, reinforcing the idea that sequence-aware representations can capture patterns that are difficult to identify from single events alone [14]. These studies do not address command-stream detection directly, and they should not be treated as equivalent to malicious command-line analysis. Even so, they provide strong methodological support for modeling chronological context in security monitoring.

This motivation is directly relevant to host-side command-line attacks. In many real incidents, malicious behavior is not expressed by one obviously dangerous command, but by a sequence of operations whose combined order reveals intent. A download step, a decoding step, and a later execution step may each appear less informative in isolation than when viewed together. This makes sequence-aware modeling a natural extension of static command detection.

Recent work has therefore improved command-line representation, strengthened single-command robustness under obfuscation, and advanced sequence-aware modeling in related security domains. However, there is still limited work that directly compares rule-based methods, traditional machine learning, and context-enhanced models within a unified framework that spans both static command classification and context-aware command-stream detection. This paper addresses that gap by evaluating these directions together and by examining how contextual modeling changes detection performance for multi-step malicious command behavior.

3 Methodology

3.1 Problem Formulation

This paper studies malicious command-line activity through two related detection tasks: single-command detection and command-stream detection. The first task is formulated as a static binary classification problem. Given an individual command line c , the objective is to predict a label $y \in \{0, 1\}$, where 0 denotes benign activity and 1 denotes malicious activity. This task models the local properties of the command itself, including its textual form and security-relevant structure.

The second task extends this setting to ordered command streams. Let $S = (c_1, c_2, \dots, c_t)$ denote a command window arranged in execution order. The objective is to predict the risk of the current step by considering not only the current command but also its surrounding context within the sequence. Unlike standard isolated text classification, this task depends on temporal order, since the meaning of a command may change when viewed together with preceding and subsequent actions. A complete detection framework should therefore support both static command analysis and context-aware stream analysis within a unified pipeline.

3.2 Overall Framework

To support these two tasks, we use a hierarchical detection framework with five stages: data construction, preprocessing, feature extraction, base classification, and context enhancement. The framework first prepares command data for either single-command or command-stream analysis. The commands are then normalized through a lightweight preprocessing stage, after which they are converted into a unified numerical representation. The complete pipeline is illustrated in Figure 1.

The first operational layer performs single-command detection using base classifiers. This layer provides an initial estimate of command-level risk from the local properties of each input. The second layer performs context enhancement by modeling ordered command windows and refining the initial predictions using temporal context. In this way, the framework separates local command analysis from sequential reasoning while keeping both stages within the same evaluation pipeline. This design enables direct comparison between static and context-aware detection methods under a consistent methodological framework.

3.3 Single-command Models

The first layer compares three models for single-command detection: rules, hybrid logistic, and random forest. These models represent three different detection strategies within the same framework.

The rule-based model acts as an interpretable baseline. It uses predefined suspicious patterns and security-relevant indicators to identify commands that

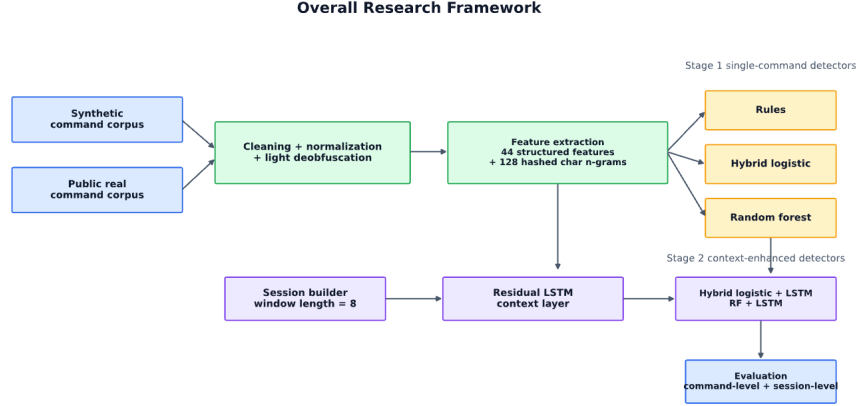


Fig. 1. Overall research framework for hierarchical malicious command-line detection

match known malicious forms. This baseline is effective because it reflects traditional command-line detection practice and provides a transparent reference point for comparison.

The hybrid logistic model acts as a linear feature-based classifier. It operates on the unified command representation and estimates risk from a weighted combination of input features. Within the framework, it provides a simple learning-based baseline that can capture broader patterns than fixed rules while remaining relatively easy to interpret.

The random forest model provides a non-linear feature-based classifier. By combining multiple decision trees, it can model feature interactions that are difficult for linear classifiers or fixed rules to capture. This makes it suitable for commands whose maliciousness depends on more complex combinations of structural and textual signals. Together, these three models provide a compact comparison between expert-driven detection, linear learning, and non-linear learning at the single-command level. Fig. 2 summarizes how these models relate to the two context-enhanced variants in the second layer.

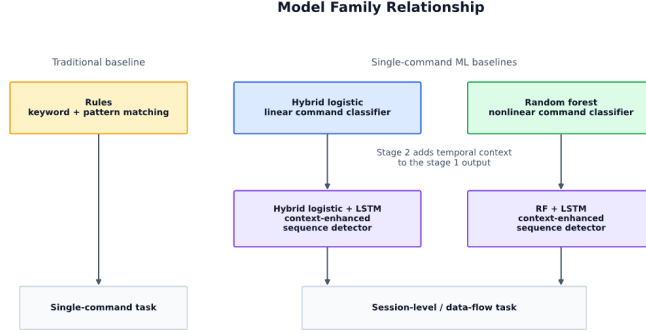


Fig. 2. Model family relationship across the two-layer framework

3.4 Context-enhanced Models

Single-command detection is effective for identifying locally suspicious inputs, but it is often insufficient for attacks that unfold across multiple steps. In many host-side scenarios, malicious intent is distributed across a sequence of commands rather than concentrated in one clearly malicious instance. To address this limitation, the second layer of the framework introduces context-enhanced models based on Long Short-Term Memory (LSTM) networks. A detailed view of RF+LSTM is provided in Fig. 3, highlighting how context-aware refinement is applied on top of first-layer RF outputs.

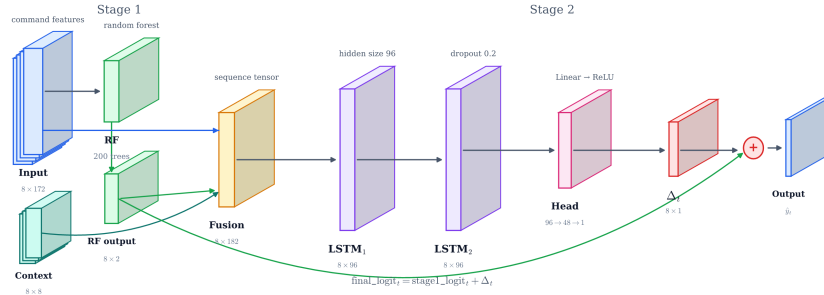


Fig. 3. RF+LSTM two-stage architecture for context-aware refinement

We implement two variants in this layer: hybrid logistic+LSTM and RF+LSTM. In both cases, the LSTM operates on ordered command windows and uses the outputs of the first-layer classifier together with command-level and contextual inputs to refine the prediction. The role of this layer is therefore not to replace the first-layer classifier, but to incorporate temporal dependencies that may help reinterpret commands whose local risk is ambiguous when viewed in isolation.

This design supports a simple hierarchical view of detection. The first layer provides a local estimate of risk for each command, while the second layer adjusts this estimate in light of surrounding activity. The resulting framework allows the same base classification logic to be examined both with and without explicit temporal context.

3.5 Input Representation

The framework uses a multi-part input representation to support both local classification and context-aware refinement. For the single-command models, each command is represented using structured security features and character n-gram features. The structured features capture security-relevant properties of the command, while the character n-gram features capture local textual variation and subword-level patterns that may remain informative under lightweight transformation or obfuscation.

For the context-enhanced layer, the representation is extended from the command level to the sequence level. In addition to the command-level representation, the LSTM layer uses first-layer outputs, such as command-level risk estimates, together with compact contextual information derived from the command window. This combination allows the model to retain local structural signals while also incorporating sequential evidence from neighboring commands.

Overall, this representation strategy is intended to match the two-layer structure of the framework. The first layer relies on a unified representation of individual commands, whereas the second layer uses this information as the basis for temporal refinement over ordered command streams.

4 Datasets and Experimental Setup

4.1 Datasets

The experimental design uses two complementary data sources to support the two detection tasks defined in this paper. For single-command detection, we use a synthetic dataset. This dataset allows controlled construction of benign commands, malicious commands, and transformed variants, making it suitable for evaluating how base classifiers respond to different command forms under a consistent setting. The use of synthetic data is motivated by controllability rather than realism: it allows the static detection layer to be examined under a structured distribution of command patterns. This design prioritizes controlled evaluation of obfuscation and transformation effects over uncontrolled real-world variability.

For command-stream detection, we use a public-source command corpus constructed from widely used offensive security and system administration resources, including Atomic Red Team, GTFOBins, LOLBAS, and MUNI KYPO datasets. These sources provide a mixture of benign administrative commands and known

malicious or dual-use command patterns, enabling evaluation under more realistic conditions than purely synthetic generation. Under this setting, the objective is not only to evaluate isolated command content, but also to test how models behave when commands are interpreted as part of an ordered stream. Public-source command material provides a more realistic basis for this task than purely synthetic generation, especially when the objective is to study how local command risk changes under surrounding context. The two-source design therefore serves two different methodological purposes: the synthetic dataset supports controlled evaluation of single-command classification, while the public-source corpus supports more realistic analysis of context-aware command-stream detection.

The dataset sources, splits, and class-scale statistics are summarized in Table 1.

Table 1. Dataset sources and scale used in this study.

Dataset	Split	Samples	Benign	Malicious	Description
Synthetic single-command	Train	156	84	72	Controlled command templates
Synthetic single-command	Test	101	52	49	Held-out transformed commands
Public real corpus	Train	6886	1401	5485	Atomic + GTFBins + LOLBAS + MUNI KYPO
Public real corpus	Test	1724	351	1373	Held-out public-source commands

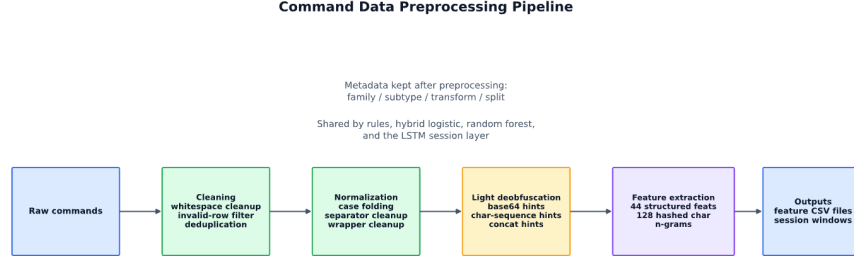
Taken together, this design balances controllability and realism within one framework. It allows the static and sequence-aware components of the proposed method to be evaluated under data conditions that are appropriate for their respective roles.

4.2 Preprocessing

Before feature extraction, all command strings undergo a lightweight preprocessing pipeline consisting of cleaning, normalization, and lightweight de-obfuscation. The purpose of this stage is to reduce superficial variation in the raw input while preserving the structural and security-relevant properties of the original command. The end-to-end preprocessing workflow is shown in Figure 4.

The cleaning step removes unnecessary formatting artifacts, such as redundant whitespace and other low-information textual irregularities, so that downstream models are less affected by accidental formatting differences. The normalization step standardizes selected variable elements when appropriate, which helps reduce dependence on transient or environment-specific strings and encourages the models to focus on recurring structural patterns. The final step applies lightweight de-obfuscation to simplify basic transformed forms that may otherwise introduce excessive surface variation without changing the underlying command intent.

This preprocessing strategy is intentionally limited in scope. It is not designed to fully recover deeply hidden malicious logic, but rather to improve the con-

**Fig. 4.** Command data preprocessing pipeline

sistency of the input space while retaining the signals needed for both static command classification and context-aware stream modeling.

4.3 Feature Design

After preprocessing, each command is converted into a unified 172-dimensional representation. This representation is used by the single-command classifiers and also serves as the basis for the context-enhanced models in the second layer. The feature design combines two complementary components: structured security features and character n-gram features. The detailed composition of this representation is summarized in Table 2.

Table 2. Feature composition of the command representation used in this study.

Feature group	Count	Description
Statistical / length features	10	Length, entropy, digit ratio, uppercase ratio, token statistics
Structure / syntax features	8	Pipe, redirect, subshell, separators, quotes, command depth
Security semantic features	17	Keywords, network, download-execute, reverse shell, dangerous flags
Deobfuscation / normalization features	9	Base64-like ratio, hex-like ratio, concat, variable wrap, decoded hints
Hashed char n-grams	128	Character 3-5 gram hashed buckets for text robustness
Total	172	Unified feature vector used by the ML models

The structured security features capture command-level properties that are relevant to security analysis. These features describe aspects such as command length, character-level irregularity, and the presence of structural patterns that may be associated with suspicious or transformed command behavior. Their purpose is to provide an explicit representation of security-relevant command characteristics in a form that can be used by both linear and non-linear classifiers.

The character n-gram features complement this view by capturing local textual variation at the subword level. Compared with word-level representations,

character n-grams are more robust to minor textual changes, delimiter variation, and lightweight obfuscation. This makes them useful for modeling commands whose suspiciousness may depend not only on high-level structure but also on local textual patterns.

The resulting representation combines explicit structural cues with fine-grained textual information. In this way, the feature design supports both interpretable security-oriented analysis and more flexible pattern learning within a common input space.

4.4 Experimental Settings

The experiments are configured to evaluate single-command detection and command-stream detection separately, while keeping both tasks within the same overall framework. For each task, the corresponding dataset is divided using a train/test split, so that all reported evaluations are performed on previously unseen data.

For the command-stream setting, the context-enhanced models operate on ordered command windows. The window size is fixed at 8, which allows the model to evaluate the current step in relation to its nearby context. This choice reflects a focus on short-range temporal dependencies typical of command execution sequences, while maintaining a consistent and computationally efficient evaluation setting. This setting reflects the methodological objective of the second layer: not to replace local command classification, but to refine it using short-range temporal dependencies across adjacent commands.

The sequence model in the second layer uses an LSTM hidden size of 96. This configuration is kept fixed across the stream-level experiments in order to provide a consistent basis for comparison between the two context-enhanced variants. The evaluation uses standard classification metrics: Accuracy, Precision, Recall, F1 score, False Positive Rate (FPR), and PR-AUC, reflecting both overall performance and the trade-off between detection sensitivity and false alarms. These metrics are selected to reflect both overall classification performance and the trade-off between detection sensitivity and false alarms, which is especially important in security monitoring settings.

Overall, the experimental setup is intended to support a direct comparison between static and context-aware detection under a common evaluation protocol. The next section reports the resulting performance of the base classifiers and the context-enhanced models under these settings.

5 Results

5.1 Single-command Detection Results

The single-command results show a clear difference among the three base models. Overall, random forest achieves the strongest overall performance under this

setting, while rules and hybrid logistic reflect two different trade-offs between conservativeness and coverage. In the current experiments, random forest achieves the highest overall accuracy and F1 score, with an accuracy of 84.16% and an F1 score of 80.95%. By comparison, hybrid logistic reaches an accuracy of 80.20%, and the rule-based model remains more conservative, with lower recall despite its low false-positive tendency. A compact model-level metric comparison is shown in Fig. 5.

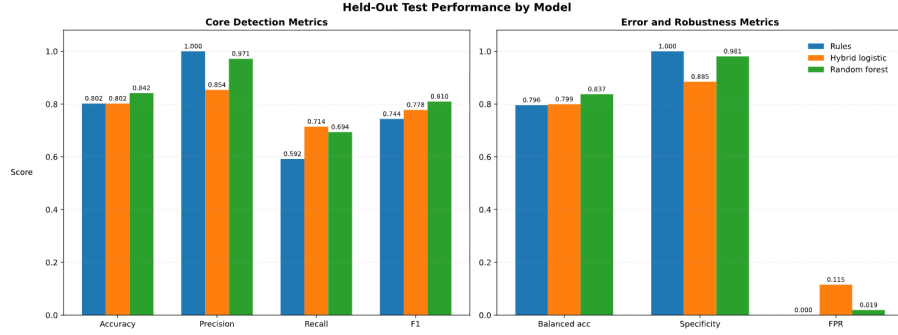


Fig. 5. Held-out test performance by model for single-command detection.

The rule-based model performs as the most conservative baseline. Its main strength lies in low false-positive behavior: in the current test setting, its reported false-positive rate is 0, which indicates that it is highly precise for the commands it flags. At the same time, this conservativeness comes at the cost of reduced attack coverage. Its recall is 59.18%, which is lower than that of the learning-based models. This pattern is consistent with the role of rules in command-line detection: they remain useful for well-known suspicious forms, but they are less effective when malicious behavior is expressed through variation, transformation, or less explicit command structure.

The hybrid logistic model improves this coverage. Its recall reaches 71.43%, which is higher than that of the rule-based baseline, indicating that the unified feature space is informative enough to support broader discrimination than exact or near-exact signature matching. This demonstrates that even a relatively simple learning-based model can capture suspicious combinations of structural and textual features that rules do not generalize to well. However, the gain in recall is accompanied by a higher false-positive tendency, indicating that the linear decision boundary is less effective at separating benign but complex command forms from genuinely malicious ones. This trade-off is also visible in the held-out confusion matrices in Fig. 6.

Among the three models, random forest provides the strongest overall balance. Compared with the linear baseline, it is better able to exploit heterogeneous hand-crafted features, including both structured security signals and character-level textual variation. This is consistent with the nature of the task: command maliciousness is often associated not with one isolated indicator, but with interactions

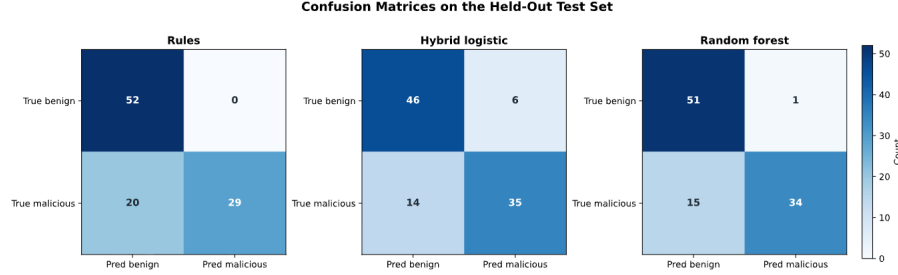


Fig. 6. Confusion matrices on the held-out test set for rules, hybrid logistic, and random forest.

among multiple local properties. The confusion-matrix analysis also supports this interpretation, showing that random forest achieves a more balanced distribution of true positives and true negatives than the other two models. Overall, the single-command results suggest that static malicious command detection benefits from non-linear modeling over the unified feature representation, and that random forest provides the strongest base classifier for the first layer of the framework. For completeness, Appendix Fig. 8 compares cross-validation averages and held-out performance.

5.2 Command-stream Detection Results

The command-stream results present a different picture from the single-command task. Once evaluation moves from isolated commands to ordered command windows, the detection problem becomes more challenging, and the value of temporal context becomes more visible. Under this setting, RF+LSTM provides the best overall performance. According to the current experiments, it achieves an accuracy of 83.69%, a precision of 69.45%, a recall of 64.52%, and an F1 score of 66.89%, making it the strongest model for stream-level detection in the current framework. The detailed session-level comparison is reported in Table 3.

Table 3. Command-stream detection performance comparison on the public dataset.

Model	TP	TN	FP	FN	Accuracy	Precision	Recall	F1
Rules	0	3574	0	1226	0.7446	0.0000	0.0000	0.0000
Hybrid logistic	675	2539	1035	551	0.6696	0.3947	0.5506	0.4598
Hybrid logistic + LSTM	562	2414	1160	664	0.6200	0.3264	0.4584	0.3813
Random forest	163	1818	1756	1063	0.4127	0.0849	0.1330	0.1037
RF + LSTM	791	3226	348	435	0.8369	0.6945	0.6452	0.6689

The results also show that static models do not transfer equally well to the stream setting. The rule-based model performs poorly in this task: although its overall accuracy remains relatively high, its recall and F1 score are both reported as 0, indicating that it identifies almost no malicious steps in the

command-stream evaluation. This is an important result in itself. In sequence-level security monitoring, high accuracy alone is not sufficient, especially when malicious windows are relatively rare. A model that defaults too strongly toward benign predictions may appear stable in aggregate while still failing to provide useful attack coverage.

The behavior of the learning-based baselines further supports the need to distinguish between static classification quality and sequence-aware detection quality. In the current experiments, the standalone random forest model performs well in the single-command task but much worse when directly applied to stream-level evaluation, where its accuracy falls to 41.27% and its F1 score to 10.37%. The hybrid logistic model performs better than standalone random forest in the stream setting, with an F1 score of 45.98%, but it still remains clearly below the best context-enhanced model. These results suggest that strong local classification is informative but not sufficient: once malicious behavior is distributed across multiple steps, the model must also use temporal context to interpret how those steps relate to one another.

The context-enhanced models make this point more explicit. The current results show that adding an LSTM layer does not improve every base model to the same extent. In particular, hybrid logistic+LSTM does not exceed its static counterpart, which demonstrates that sequence modeling is most effective when the first-layer classifier already provides informative local signals. In contrast, RF+LSTM combines strong command-level discrimination with temporal refinement, allowing the model to reinterpret locally ambiguous commands through surrounding context. This is consistent with the intended role of the second layer in the proposed framework: not to replace the base classifier, but to refine its local estimates when neighboring steps provide additional evidence. These error patterns across models are further illustrated in Fig. 7.

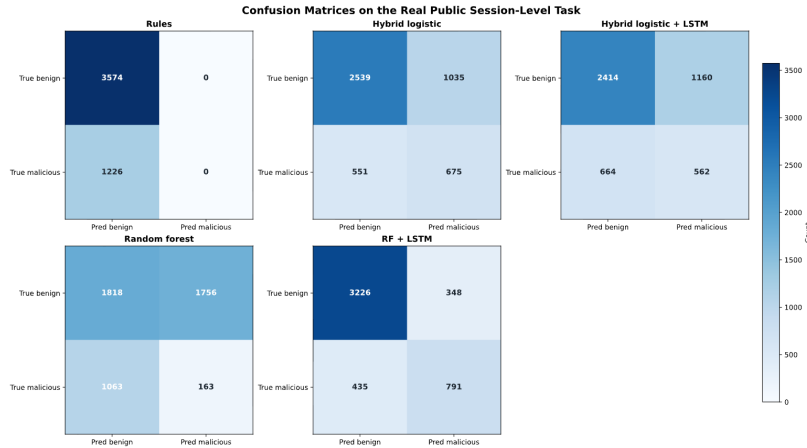


Fig. 7. Confusion matrices on the real public session-level task for all evaluated models.

Taken together, the stream-level results support two conclusions. First,

command-stream detection should not be evaluated in the same way as isolated command classification, because the temporal distribution of malicious intent changes the task substantially. Second, context enhancement is most useful when the base model already offers a reliable local representation of risk. In the current study, this combination is best realized by RF+LSTM, which provides the strongest overall balance across stream-level metrics and illustrates the value of hierarchical detection for multi-step malicious command behavior. This result highlights a key limitation of static detection: strong single-command performance does not translate to sequence-level effectiveness without explicit context modeling.

6 Discussion and Limitations

6.1 Discussion

The results highlight a clear distinction between static command classification and sequence-aware threat detection. Effective malicious command-line detection depends on two complementary capabilities: reliable local discrimination at the single-command level and context-aware refinement at the command-stream level.

For single-command detection, random forest achieves the strongest overall performance. This outcome is consistent with the feature design used in this study, where command maliciousness depends on interactions among multiple structural and textual features. In this setting, non-linear models are better suited to capture complex feature relationships than rule-based or linear approaches. The rule-based model remains useful as a conservative baseline with low false-positive behavior, while hybrid logistic regression improves recall but remains limited by its linear decision boundary.

In contrast, command-stream detection introduces additional complexity, as malicious intent is often distributed across multiple steps rather than concentrated in a single command. Under this setting, the performance of RF+LSTM indicates that temporal context plays a critical role when local signals alone are insufficient. The LSTM layer does not replace the base classifier, but refines its predictions by incorporating sequential dependencies across commands.

The weaker performance of hybrid logistic+LSTM further suggests that context enhancement is most effective when the base classifier provides stable and informative local predictions. Overall, these findings support a hierarchical view of detection, where local command classification and temporal modeling serve complementary roles rather than interchangeable ones.

6.2 Limitations

The results of this study should be interpreted within several methodological constraints.

First, the single-command experiments rely on a synthetic dataset. While this enables controlled evaluation of command transformations and obfuscation

effects, it does not fully reflect the variability and noise present in real-world command-line activity.

Second, the command-stream evaluation is based on fixed-length windows rather than complete execution sessions. This provides a controlled approximation of temporal context, but does not capture longer or irregular attack sequences observed in practice.

Third, the framework focuses on command content and sequence structure only. It does not incorporate additional contextual signals such as process lineage, user context, timing behavior, or system state, which are commonly used in operational detection systems.

Finally, all experiments are conducted under offline evaluation. Deployment-related factors, including runtime performance, system integration, and alert handling, are not considered in this work.

7 Conclusion

This paper investigated malicious command-line detection at two levels: single-command classification and command-stream analysis. The primary contribution is a unified hierarchical evaluation framework that enables direct comparison between rule-based methods, feature-based machine learning models, and context-enhanced sequence models within a consistent setting.

The results show that static and sequence-aware detection should be treated as related but distinct tasks. Random forest achieves the strongest performance for single-command detection, while RF+LSTM provides the best results for command-stream analysis. This indicates that strong local classification is necessary but insufficient when malicious behavior is distributed across multiple steps.

Overall, the findings show that effective command-line threat detection benefits from combining local discrimination with context-aware modeling. The proposed framework provides a structured basis for understanding how temporal context influences detection performance in multi-step attack scenarios.

Appendix A Supplementary Results

This appendix provides additional result visualizations for Section 5. Fig. 8 compares cross-validation averages with held-out test performance for single-command detection. Fig. 9 reports an alternative metric-level comparison for the session-level task, and Fig. 10 gives an explicit TN/FP/FN/TP quadrant view.

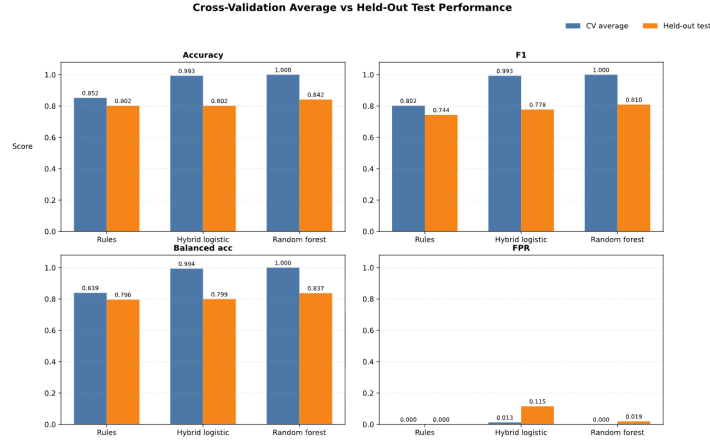


Fig. 8. Cross-validation average versus held-out test performance for single-command models.

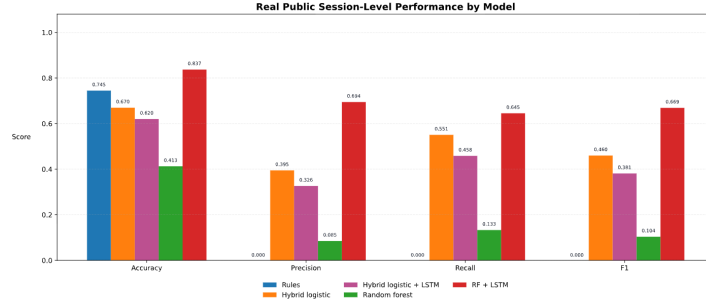


Fig. 9. Real public session-level performance by model.

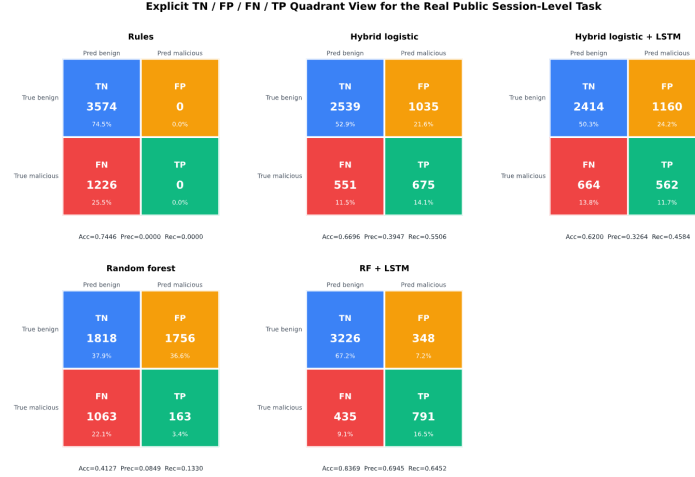


Fig. 10. Explicit TN/FP/FN/TP quadrant view for the real public session-level task.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. MITRE ATT&CK: Command Obfuscation: T1027.010. Accessed: 2026-03-29 (2025). <https://attack.mitre.org/techniques/T1027/010/>
2. Ongun, T., Stokes, J.W., Bar Or, J., Tian, K., Tajaddodianfar, F., Neil, J., Seifert, C., Oprea, A., Platt, J.C.: Living-off-the-land command detection using active learning. In: Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses. RAID '21, pp. 442–455. ACM, San Sebastian, Spain (2021). <https://doi.org/10.1145/3471621.3471858>
3. Hendler, D., Kels, S., Rubin, A.: Detecting malicious PowerShell commands using deep neural networks. In: Proceedings of the 2018 ACM Asia Conference on Computer and Communications Security. AsiaCCS '18, pp. 187–197. ACM, Incheon, Republic of Korea (2018). <https://doi.org/10.1145/3196494.3196511>
4. Alahmadi, A., Alkhraan, N., BinSaeedan, W.: MPSAutodetect: A malicious PowerShell script detection model based on a stacked denoising auto-encoder. Computers & Security **116**, 102658 (2022) <https://doi.org/10.1016/j.cose.2022.102658>
5. Fu, Z., Song, L., Ding, S.H.H., Alaca, F., Acharya, S.: Toward a robust detection of PowerShell malware against code mixing and obfuscation by

- using sentence transformer and similarity learning. *ACM Transactions on Privacy and Security* **28**(4), 55 (2025) <https://doi.org/10.1145/3771542>
6. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. CCS '17, pp. 1285–1298. ACM, Dallas, TX, USA (2017). <https://doi.org/10.1145/3133956.3134015>
 7. Himler, P., Landauer, M., Skopik, F., Wurzenberger, M.: Towards detecting anomalies in log-event sequences with deep learning: Open research challenges. In: *Proceedings of the 2023 European Interdisciplinary Cybersecurity Conference*. EICC '23, pp. 71–77. ACM, Stavanger, Norway (2023). <https://doi.org/10.1145/3590777.3590789>
 8. Zhu, T., Zheng, J., Chen, T., Lv, M., Xiong, C., Weng, Z., Zheng, X.: LOTLDetector: Living off the land attacks detection system based on feature fusion. *Cybersecurity* **9**(1), 4 (2026) <https://doi.org/10.1186/s42400-025-00531-w>
 9. Huang, S.-Y., Yang, C.-L., Lin, C.-Y., Huang, C.-Y.: CmdCaliper: A semantic-aware command-line embedding model and dataset for security research. In: Al-Onaizan, Y., Bansal, M., Chen, Y.-N. (eds.) *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 20188–20206. Association for Computational Linguistics, Miami, Florida, USA (2024). <https://doi.org/10.18653/v1/2024.emnlp-main.1126>
 10. Hussain, Z., Nurminen, J.K., Ranta-aho, P.: Training a language model to learn the syntax of commands. *Array* **23**, 100355 (2024) <https://doi.org/10.1016/j.array.2024.100355>
 11. Li, R., Zhang, C., Chai, H., Ying, L., Duan, H., Tao, J.: PowerPeeler: A precise and general dynamic deobfuscation method for PowerShell scripts. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. CCS '24. ACM, Salt Lake City, UT, USA (2024). <https://doi.org/10.1145/3658644.3670310>
 12. Zhang, S., Li, S., Lu, J., Yang, W.: Power-ASTNN: A deobfuscation and ast neural network enabled effective detection method for malicious PowerShell scripts. *Computers & Security* **154**, 104441 (2025) <https://doi.org/10.1016/j.cose.2025.104441>
 13. Gökstorp, S.G.E., Nyberg, J., Kim, Y., Johnson, P., Dán, G.: Anomaly detection in security logs using sequence modeling. In: *2024 IEEE/IFIP Network Operations and Management Symposium*. NOMS 2024, pp. 1–9. IEEE, Seoul, South Korea (2024). <https://doi.org/10.1109/NOMS59830.2024.10575561>

14. Chai, X., Zhang, H., Zhang, J., Sun, Y., Das, S.K.: Log sequence anomaly detection based on template and parameter parsing via BERT. *IEEE Transactions on Dependable and Secure Computing*, 1–18 (2024) <https://doi.org/10.1109/TDSC.2024.3428538>